

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE CODE: ITA 0614

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:50

Annexure A

Comparative Analysis

Code of Conduct:

I M. PRANATI (192312298) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

A. Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are both used for function approximation, but they differ in how they use local data.

Locally Weighted Regression

LWR gives **higher importance to training points that are closer to the query point** and lower importance to points that are farther away. A weighting function, commonly a Gaussian function, is used.

For example, consider the relationship between **temperature and electricity consumption**. To predict electricity consumption at 32°C, LWR gives more importance to observations at 30°C and 35°C than to observations at 20°C.

The prediction can be represented as:

$$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}$$

where w_i is the weight assigned according to the distance from the query point.

RBF Network

An RBF network uses **radial basis functions** as hidden-layer neurons. Each neuron has a centre, and its activation depends on the distance between the input and the centre.

A common Gaussian RBF is:

$$\phi(x) = e^{-\frac{\|x-c\|^2}{2\sigma^2}}$$

where c is the centre and controls the width of the function.

Feature	Locally Weighted Regression	RBF Network
Locality	Gives higher weight to training points close to the query point	Uses radial basis functions centred at selected points
Training	Very little or no explicit training; computation occurs during prediction	Requires training to determine centres and weights
Prediction cost	High, because nearby points must be considered for every query	Lower after training
Smoothness	Depends strongly on the bandwidth; small bandwidth gives fewer smooth results	Usually produces smooth predictions because RBFs overlap
Adaptability	Easily adapts when new data is added	New data generally requires retraining or updating weights

Example: Consider a dataset containing temperature and electricity consumption:

Temperature (°C)	Consumption
20	40
25	50
30	65
35	85
40	110

If we want to predict consumption at **32°C**, LWR gives more importance to points around 32°C, such as 30°C and 35°C.

An RBF network places basis functions around representative data points or centers and combines their outputs to predict consumption.

Conclusion: LWR is highly flexible and locally adaptive but computationally expensive during prediction. RBF networks require more initial training but provide faster and generally smoother predictions.

3. Analyse the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

A.K-Nearest Neighbour (KNN) and RBF networks can both classify data based on similarity, but their decision-making processes are different.

K-Nearest Neighbour (KNN) and **Radial Basis Function (RBF) networks** are supervised learning methods used for classification. Both use the similarity or distance between data points, but they create classifications in different ways.

K-Nearest Neighbour

KNN is a **lazy learning algorithm** because it does not build an explicit model during training. It stores the training data and, when a new sample arrives, calculates its distance from the training samples.

The most common distance is Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

The **K nearest samples** is selected and majority voting is used for classification.

For example, if 4 nearby emails are spam while 1 is not spam, the new email is classified as **spam**.

RBF Network

An RBF network consists of:

1. Input layer
2. Hidden layer containing RBF neurons
3. Output layer

The hidden neurons measure the distance between an input and their centres. A Gaussian function is commonly used.

The network learns the output weights during training.

Training Time

KNN has almost no training phase.

Therefore:

Training time = very low

But classification can be slow because distances to many training samples must be calculated.

RBF requires training to find suitable:

- Centres
- Widths
- Output weights

Therefore, its training time is higher.

Generalization

KNN's generalization depends strongly on the value of K.

- Very small K → sensitive to noise and overfitting.
- Very large K → may ignore important local patterns and underfit.

Feature	KNN	RBF Network
Training time	Very low; mainly stores training data	Higher because centres and weights must be learned
Classification	Uses classes of the K nearest samples	Uses weighted activation of RBF neurons
Decision boundary	Can be irregular and highly dependent on training samples	Usually, smoother
Prediction time	High for large datasets because distances must be calculated	Generally faster after training
Generalization	Can overfit when K is too small	Usually, better smooth generalization
New data	Easily incorporated without retraining	May require model updating

Example: Suppose we classify emails as **Spam** or **Not Spam** using features such as number of links and suspicious words.

- KNN finds the nearest training emails and assigns the class based on majority voting.
- An RBF network transforms the input through radial basis functions and produces a class using learned output weights.

For example, if **K = 3** and two of the three nearest emails are spam, KNN classifies the new email as **Spam**.

Decision boundary: KNN may create a complex, irregular boundary around individual training samples, whereas an RBF network normally creates a smoother boundary.

Conclusion: KNN is simple and useful for small datasets with continuously arriving data. RBF networks require more training but can provide faster prediction and smoother decision boundaries for larger classification problems.

4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.

It solves a new problem by finding similar previous cases and reusing their solutions. Locally Weighted Regression (LWR) predicts a continuous value by giving greater importance to nearby data points.

Case-Based Learning (CBL) and **Locally Weighted Regression (LWR)** are learning approaches that use previously observed examples to solve new problems.

The major difference is:

CBL → retrieves and adapts previous solutions

LWR → uses nearby examples to calculate a numerical prediction.

Case-Based Learning

Case-Based Learning stores previous problems and their solutions as **cases**.

When a new problem occurs, the system:

1. Retrieves similar cases.
2. Selects the most relevant case.
3. Reuses or modifies its solution.
4. Stores the new experience for future use.

This is often described as:

Retrieve → Reuse → Revise → Retain

Example: Recommendation System

Suppose an online learning platform has thousands of students.

For each student, the system stores:

- Skills
- Interests
- Previous courses
- Course ratings
- Completed courses
- Learning behaviour

Suppose a new student has strong Python skills and is interested in Machine Learning.

CBL searches for previous students with similar profiles.

If similar students successfully completed:

- Machine Learning
- Data Science
- Deep Learning

these courses can be recommended to the new student.

Locally Weighted Regression

LWR predicts an output using nearby examples.

Suppose we want to predict the **rating a student will give to a course**.

The system considers:

- Similarity in skills
- Previous ratings
- Study time
- Course difficulty
- Previous performance

Students similar to the new student receive higher weights.

Case-Based Learning:

- Stores previous students and their course choices.
- Finds students similar to a new student.
- Recommends courses that worked well for those similar students.

Locally Weighted Regression:

- Uses features such as study hours, skill level, course rating, and previous scores.
- Gives higher weights to students with similar feature values.
- Predicts the expected rating or interest for a course.

Feature	Case-Based Learning	LWR
New data	Easily adds new cases to the case library	Can directly use new data in future predictions
Adaptability	Very high; retains previous cases	High; prediction changes with local data
Computation	Searching for similar cases can be expensive	Prediction can be expensive because local weights are calculated
Output	Usually retrieves/adapts a previous solution	Produces a numerical prediction
Best suited for	Problems with reusable past experiences	Continuous prediction problems

Conclusion: CBL is suitable when previous solutions can be directly reused, while LWR is better when we need a numerical prediction based on nearby examples.

6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyse assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

Both **Naïve Bayes (NB)** and **Logistic Regression (LR)** are commonly used for text classification such as spam detection.

Introduction

Spam detection is a common **text classification** problem. The objective is to classify an email as:

- **Spam**
- **Not Spam (Ham)**

Naïve Bayes and Logistic Regression are both effective methods for this task.

Naïve Bayes

Naïve Bayes is a probabilistic classifier based on **Bayes' theorem**.

For spam detection:

- = Spam or Ham
- = words/features in the email

The algorithm estimates how likely the observed words are for each class.

Feature Independence Assumption

Naïve Bayes assumes that features are **conditionally independent given the class**.

For example, it assumes that words:

- "free"
- "offer"
- "winner"

contribute independently to the spam probability.

This assumption is not always true in real text, but Naïve Bayes can still work very well.

Logistic Regression

Logistic Regression predicts the probability of a class using a sigmoid function.

where:

Each word is represented as a feature, and the model learns a weight for each feature.

For example:

- "free" → positive spam weight
- "winner" → positive spam weight
- "meeting" → possibly negative spam weight

The total score determines whether the email is spam.

Example

Consider:

"You won a FREE prize. Click now to claim your OFFER."

The words **free**, **prize**, **claim**, **offer** may strongly indicate spam.

Naïve Bayes calculates the probability using the frequency of these words in spam and ham emails.

Logistic Regression combines the learned weights of these words and produces a spam probability.

Feature	Naïve Bayes	Logistic Regression
Feature assumption	Assumes features are conditionally independent given the class	Does not require feature independence
Probability estimation	Estimates $P(\text{Spam} \text{Words})$ using Bayes' theorem	
Training speed	Very fast	Fast, but usually more computationally expensive
High-dimensional data	Performs well with many text features	Also performs very well with sparse high-dimensional text
Data requirement	Works well with relatively small datasets	Usually benefits from more training data
Feature correlation	Correlated features can reduce effectiveness	Handles correlated features better

Example

Suppose an email contains words such as:

"Congratulations", "free", "prize", "offer"

Naïve Bayes calculates the probability of the email being spam based on the probabilities of these words occurring in spam and non-spam emails.

Logistic Regression combines the weighted features:

If the probability is greater than 0.5, the email can be classified as spam.

When does each perform better?

- **Naïve Bayes:** Often performs well when the dataset is small and contains many independent word features.
- **Logistic Regression:** Can outperform NB when features are correlated and sufficient training data is available.

Conclusion: Naïve Bayes is fast, simple, and effective for text classification, while Logistic Regression is more flexible and does not require the feature-independence assumption. For a small and simple spam dataset, Naïve Bayes may be preferred. For a larger dataset with complex feature relationships, Logistic Regression may provide better performance.

10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

Principal Component Analysis (PCA) and Feature Selection reduce dimensionality, but they do so in different ways.

Introduction

Machine learning datasets may contain hundreds or thousands of features. A very large number of features can increase computation, storage requirements, and the possibility of overfitting.

Two common dimensionality-reduction approaches are:

1. **Principal Component Analysis (PCA)**
2. **Feature Selection**

Both reduce the number of dimensions, but they work differently.

Principal Component Analysis

PCA is a **feature transformation technique**.

It converts the original correlated features into a smaller number of new variables called **principal components**.

The components are ordered according to the amount of variance they explain.

For example:

$$X_1, X_2, X_3, \dots, X_{100}$$

can be converted into:

$$PC_1, PC_2, \dots, PC_{10}$$

If the first 10 components contain 95% of the variance, the remaining components may be discarded.

Important Properties of PCA

- Reduces dimensionality
- Removes correlation between components
- Preserves maximum variance
- Creates new features
- Can reduce computational complexity

Feature Selection

Feature Selection does not create new features.

Instead, it identifies and keeps the **most useful original features**.

For example, from 100 features, it may select:

- Attendance
- Previous marks
- Study hours
- Assignment score

- Internal marks

The other 95 features are removed.

Types of Feature Selection

Feature selection methods include:

1. Filter methods

Use statistical measures such as:

- Correlation
- Chi-square
- Mutual information

2. Wrapper methods

Evaluate different subsets using a machine learning model.

Examples:

- Forward selection
- Backward elimination

3. Embedded methods

Feature selection occurs during model training.

Examples:

- Lasso regression
- Decision trees

Example

Suppose a student-performance dataset contains **50 features**.

Some features may be:

- Attendance
- Study hours
- Previous marks
- Assignment score
- Age
- Internet usage
- Number of social-media posts
- Random ID number

Using PCA

$$PC1 = 0.6X_1 + 0.5X_2 + 0.4X_3 + \dots$$

PCA combines the original variables and creates components such as:

The final model may use 5 principal components instead of 50 features.

Using Feature Selection

Feature Selection may identify:

- Attendance
- Study hours
- Previous marks

- Assignment score
- Internal marks

and discard irrelevant features.

Information Preservation

PCA attempts to preserve **maximum variance**.

For example:

$$100 \text{ features} \rightarrow 10 \text{ components}$$

If those components preserve 95% variance, only a small amount of variation is lost.

Feature Selection preserves the exact values of selected features but may lose useful information if an important feature is removed.

Computational Complexity

PCA involves operations such as:

- Computing covariance/correlation
- Eigenvalue/eigenvector calculation or singular value decomposition

Therefore, PCA can be computationally expensive for very large datasets.

Feature Selection is often computationally cheaper, especially with simple filter methods.

Interpretability

This is one of the biggest differences.

PCA

$$PC1 = 0.4(\text{attendance}) + 0.5(\text{marks}) + 0.3(\text{study hours})$$

Low interpretability because each component may be a combination of many original variables.

For example:

It is difficult to give PC1 a simple real-world meaning.

Feature Selection

High interpretability because the model uses the original features.

For example:

Attendance → 80%

is easy to understand.

Effect on Downstream Models

PCA can improve the performance of algorithms affected by:

- High dimensionality
- Feature correlation
- Noise

It is particularly useful with algorithms such as:

- KNN
- SVM
- Logistic Regression

Feature Selection can improve:

- Training speed
- Model interpretability
- Generalization
- Memory usage

It can also reduce overfitting by removing irrelevant features.

Feature	PCA	Feature Selection
Method	Creates new features called principal components	Selects existing important features
Information preservation	Preserves maximum variance in fewer components	May lose information if useful features are removed
Interpretability	Lower because components combine many features	High because original features are retained
Computation	Can be computationally expensive for large datasets	Usually simpler and faster
Feature meaning	Original meaning is lost	Original meaning is retained
Model performance	Can improve performance by removing redundancy/noise	Can improve performance by removing irrelevant features

Example

Suppose a dataset contains **100 features** for predicting student performance.

Using **PCA**, the 100 features may be transformed into 10 principal components:

These 10 components contain most of the important variation in the original dataset.

Using **Feature Selection**, we may instead select 10 original features such as:

- Attendance
- Study hours
- Previous marks
- Assignment scores
- Test scores

Effect on downstream ML

- **PCA:** Reduces dimensionality and correlation, which can make algorithms such as KNN and Logistic Regression faster.
- **Feature Selection:** Removes irrelevant features while keeping meaningful information, making the final model easier to explain.

Computational Complexity

PCA can require significant computation because it involves operations such as:

1. Standardizing the data
2. Calculating the covariance matrix
3. Finding eigenvalues and eigenvectors or using Singular Value Decomposition (SVD)
4. Selecting principal components
5. Transforming the dataset

Therefore, PCA can be computationally expensive for very large datasets.

Feature Selection can be less computationally expensive, especially when using simple **filter methods**.

For example, we can calculate the correlation between each feature and the target and select the most relevant features.

However, wrapper-based feature selection can become computationally expensive because many feature combinations may need to be evaluated.

Conclusion: PCA and Feature Selection both reduce dimensionality but follow different approaches. PCA transforms the original features into new components and focuses on preserving variance, whereas Feature Selection chooses the most relevant original features. PCA is useful when the dataset has many correlated variables, while Feature Selection is preferred when interpretability and the original meaning of features are important.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand